



PROJECT-BASED LEARNING

PHASE-I EVALUATION

SKIP LIST

Team ID: DSCPP-III-2026-T049

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026-27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T049
Semester: 3rd
Section: DS1,ML2,CYBER,ML1
Domain: *DSCPP*

Team Members

1. *Sarthak Gera – 2510380077*
2. *Rudraksh Gera – 2510350038*
3. *Arjun Bhadana – 2510370135*
4. *Divyansh Silswal – 2510370989*

Mentor

Ms. Priyanka Rastogi

Interactions completed: 3

Problem Statement & Motivation

Problem Statement

Design and Implementation of a SKIP LIST–Based Key-Value Store

Motivation

- ▶ *No simple data structure offers both fast search and fast insertion on sorted data.*
- ▶ *AVL/Red-Black trees guarantee $O(\log n)$ but their rotation logic is complex and error-prone to implement.*
- ▶ *Skip Lists offer a simpler, randomised alternative*

Target Users / Stakeholders

- ▶ *Developers needing a fast, ordered key-value store*
- ▶ *CS students learning Skip Lists as a simpler AVL/RB-tree alternative*
- ▶ *Easier to implement $O(\log n)$ performance without complex rotations*

Objectives

Project Objectives

1. *Design and implement a templated Skip List in C++ from scratch.*
2. *Support insert, search, delete, and display in $O(\log n)$ time.*
3. *Apply OOP: templates, encapsulation, constructors*
4. *Evaluate the system using suitable metrics or test cases.*
5. *Demonstrate practical applicability and future scope.*

Key Objective: *Prove Skip Lists match AVL-tree performance with far simpler implementation*

Proposed Solution

Proposed Approach

1. *Input:CLI accepts SET/GET/DEL/RANGE commands*
2. *Processing:Parser extract key,value,args*
3. *System Logic:KeyValueStore calls SkipList engine*
4. *Storage:Nodes with multi-level forward pointers*
5. *Output:Benchmark vs std::map,unordered map*

Key Features

- ▶ *Templated SkipList class*
- ▶ *Randmoized leveling, $O(\log n)$ oops*
- ▶ *SET/GET/DEL/RANGE commands*
- ▶ *Benchmarked against STL containers*
- ▶ *OOP: encapsulation, constructor, overloading*

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	<i>Pointers, Linked Lists, Trees/BST, Searching Recursion, Complexity Analysis</i>	<i>Module X: SkipList Core Engine</i>
OOPs with C++	<i>Classes, Templates, Constructors/Deconstructors, Operator Overloading, STL</i>	<i>Module Y: KeyVal-ueStore Layer</i>

Requirement: Demonstrate meaningful integration of both subjects in **one** project.

System Architecture / Workflow



Major Components

- ▶ *CLI+ Command Parser(reads SET/GET/DEL/RANGE)*
- ▶ *KeyValueStore class(routes commands)*
- ▶ *SkipList T Engine(Node, Random Level, Memory Manager)*
- ▶ *Benchmark Module (compares vs std::map, unorderedmap)*

Data / Control Flow

- ▶ *Data moves from CLI through Parser, KeyValueStore, into SkipList T*
- ▶ *KeyValueStore calls SkipList insert/search/delete via Node Pointers*
- ▶ *Correct SET/GET/DEL/RANGE results, validated against benchmarks*

Technology Stack & Methodology

Technology Stack

- ▶ Programming: *C / C++17*
- ▶ Database: *N/A(in-memory only)*
- ▶ Tools / Framework: *STL, g++*
- ▶ IDE: *VS Code*
- ▶ Version Control: *Git / GitHub*

Development Methodology

1. Requirement Analysis
2. Skip List
Design(insert/search/delete)
3. Templating and OOP Generalisation
4. KeyValueStore Implementation
5. Benchmarking and Testing

Justify the selection of technologies and methodology.

Initial Progress & Team Contribution

Progress Achieved

- ▶ *Literature study on Skip Lists vs AVL/Red-Black trees*
- ▶ *Requirements identified (SET/GET/DEL/RANGE operations)*
- ▶ *SkipList T and Node Architecture designed*
- ▶ *Initial Node structure and pointer logic drafted*
- ▶ *GitHub repository initialised for version control*

Individual Contribution

Member	Role	Contribution
Sarthak Gera	Lead/OOP	SkipList T class design, templates, encapsulation
Arjun Bhadana	Developer	KeyValueStore implementation, CLI command parser
Divyansh Silswal	DB/Testing	Benchmarking module, testing vs std::map/unordered map
Rudraksh Gera	Developer/Docs/ DSA	Node structure, memory management, documentation

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal & Design
2. Phase-II: Development
3. Phase-III: Final Implementation

Expected Outcomes

- ▶ *Fully function templated Skip List(insert, search, delete, display)*
- ▶ *$O(\log n)$ performance validated through benchmarking*
- ▶ *working Key Value Store with SET/GET/DEL/RANGE*
- ▶ *Practical alternative to AVL/Red-Black trees*

References

1. *Pugh, W.(1990). Skip Lists:A Probabilistic Alternative to Balanced Trees. CACM,33(6)*
2. *Redis Docs-Sorted Sets*
3. *GeeksforGeeks-Skip List*
4. *LevelDB Documentation - Memtable*

Thank You
Questions & Suggestions